



AI Purchasing Agent — Assignment

Background

A retail / quick-commerce company manages purchasing of products from multiple suppliers across multiple fulfillment nodes.

Today, buyers spend significant time manually reviewing purchasing information and making decisions about what, when, and how much to purchase.

Some of the information available to buyers may include inventory, demand, forecasts, existing purchase orders, supplier conditions, budgets, storage constraints, and other operational information.

We want to explore whether an **AI agent can automate part of the buyer's workflow**.

Your Task

Build a **full-stack AI Purchasing Agent** that can assist a buyer in making purchasing decisions.

The agent should be able to take a purchasing situation, investigate the relevant information, make an appropriate decision, and take action where appropriate.

The purchasing recommendation provided to the agent should **not necessarily be assumed to be correct**.

The agent should be able to deal with situations where multiple factors or constraints affect the final decision.

You are free to determine how the agent should solve the problem.

Scenarios

Scenario 1 — Purchase Recommendation Review

The purchasing system recommends buying **800 units** of a product.

The buyer's available information includes:

- current inventory;
- expected demand;
- existing open purchase orders;
- supplier lead time;
- supplier minimum order requirements;
- available purchasing budget;
- available storage capacity.

The agent needs to determine whether the recommendation should be:

- accepted;
- modified;
- rejected; or
- investigated further.

The agent should explain the important factors behind its decision.

Scenario 2 — Supplier Cannot Fulfil the Purchase

A purchase order has been created for **500 units**, but the supplier informs the system that only **250 units** can currently be supplied.

The agent needs to determine what should happen next.

Consider what additional information might be required before taking action.

For example, the agent may need to determine whether:

- the remaining quantity should be sourced elsewhere;
- another supplier should be considered;
- an additional purchase order is required;
- the existing inventory is sufficient;
- or the situation should be escalated.

You should decide what information and capabilities your agent needs.

Scenario 3 — Demand / Forecast Has Changed

A product was expected to have relatively normal demand, but actual sales have increased significantly.

There is an existing purchase order, but the current inventory and incoming quantity may no longer be sufficient to cover expected demand.

The agent should investigate the situation and determine whether the purchasing plan needs to change.

The agent should consider what evidence is required before making the decision.

Scenario 4 — Purchasing Constraint

The agent determines that additional inventory should be purchased, but one or more constraints may prevent the recommended purchase from being executed.

The agent should determine an appropriate course of action rather than blindly executing the original recommendation.

Agent Behaviour

The scenarios above intentionally do not specify how the agent should solve the problem.

You should determine:

- what information the agent needs;
- what tools or APIs it should use;
- what data should exist;
- how the agent should interact with those tools;
- how decisions should be made;
- what actions the agent should be allowed to perform;
- when human approval may be appropriate;
- and how the result should be validated.

You may create **mock data, APIs, databases, or services** as necessary.

There is no prescribed architecture or technology stack.

Feedback and Validation

Your solution should demonstrate that the agent can **validate the result of its decision or action**.

For example, if the agent decides to create or modify a purchase order, your system should have a way to determine whether the resulting purchase order is actually acceptable.

If the outcome is different from what the agent expected, the system should be able to handle that situation appropriately.

We are particularly interested in how you design this **feedback loop**.

Evaluation

Create a small set of test scenarios demonstrating how your agent behaves.

Your evaluation should show how you determine whether the agent's decisions are appropriate.

Consider:

- Was the decision correct?
- Did the agent obtain the necessary information?
- Did it respect relevant constraints?
- Did it take the appropriate action?
- Did it validate the result?
- What happens when the initial action does not work?

You do not need to build a sophisticated evaluation framework.

We are more interested in the **quality of your evaluation approach**.

Scope

The assignment is intended to take approximately one working day / 6–8 hours.

You are not expected to implement all four scenarios.

At least one scenario must be implemented end-to-end.

A strong solution may support more than one scenario, but breadth is less important than the quality of the core solution.

You are also not expected to build production-grade infrastructure.

Mock APIs, databases, and data are completely acceptable.

Technology

You may choose any:

- programming language;
- frontend/backend framework;
- database;
- LLM/provider;
- AI agent framework;
- infrastructure.

AI-assisted development tools such as ChatGPT, Claude, Cursor, GitHub Copilot, etc. are allowed. You should be able to explain and modify the solution during the evaluation discussion.

Submission

Please create a **public GitHub repository** for your assignment and submit the repository URL.

The repository must contain:

- Complete source code
- `README.md` with setup and run instructions
- Architecture diagram
- Description of your approach
- Test scenarios and evaluation approach
- Any mock APIs, datasets, or supporting services required to run the application
- Explanation of how you validate the agent's decisions
- Working demo
- `.env.example` for required environment variables
- Commits after deadline won't be considered.

Do not commit API keys, passwords, tokens, or any other secrets.

Please keep your **Git commit history intact**, as we may review it as part of the evaluation.

Optional

If you identify additional buyer problems that could be solved using the same agent or architecture, feel free to demonstrate them.

For example, you may identify opportunities around:

- open purchase orders;
 - supplier reliability;
 - alternate suppliers;
 - promotional buying;
 - forecast anomalies;
 - replenishment;
 - safety stock;
 - seasonal demand;
 - or another problem you discover yourself.
-

What We Are Looking For

There is no single correct solution.

We are interested in how you approach an ambiguous real-world problem, including:

- how you break down the problem;
- what information you believe is important;
- how you design the agent;

- what tools and APIs you create;
- how the agent interacts with other systems;
- how you handle constraints and uncertainty;
- how you design feedback loops;
- how you validate agent decisions;
- and how you make the system reliable.

The goal is not to build a chatbot that answers purchasing questions. The goal is to demonstrate a system capable of making, executing, and validating purchasing decisions.